Phase 9: M-ary and Strict M-ary Trees

0. Purpose of This Phase

Phase 9 teaches how binary tree ideas grow into **M-ary tree** ideas.

Until now, most of the curriculum focused on **binary trees**. A binary tree allows each node to have at most **2 children**: a left child and a right child.

In Phase 9, we move from this:

```
Binary tree: at most 2 children per node
```

To this:

```
M-ary tree: at most M children per node
```

The main beginner idea is:

> A binary tree is only one special type of M-ary tree where M = 2.

So Phase 9 helps you understand trees where each node may have 3 children, 4 children, 5 children, or any fixed number M.

---

1. Current Progress Analysis

Before learning Phase 9, you already studied the binary tree foundation.

Phase	What you learned
Phase 1	Tree vocabulary: root, parent, child, leaf, internal node, degree, level, height
Phase 2	Binary tree formulas, height-node relationships, Catalan numbers, degree-0 and degree-2 relationship
Phase 3	Binary tree representation using arrays, linked nodes, queues, and pointers
Phase 4	Recursive traversals: preorder, inorder, postorder, and visual traversal shortcuts

Phase	What you learned
Phase 5	Iterative traversals using stacks and level-order traversal using queues
Phase 6	Recursive tree analysis: count nodes, count degree-2 nodes, sum values, calculate height
Phase 7	Strict binary trees: every node has either 0 or 2 children
Phase 8	Difference between strict, full, and complete binary trees

So your current position is:

> You understand binary trees well enough to now generalize the same ideas to trees where each node can have up to M children.

Phase 9 covers these main topics:

1. What an M-ary tree is.
2. Difference between binary trees and M-ary trees.
3. How to represent an M-ary tree node in C++.
4. What a strict M-ary tree is.
5. How to check whether an M-ary tree is strict.
6. Important formulas for strict M-ary trees.
7. Internal and external node relationship in strict M-ary trees.
8. Complete beginner C++ program.
9. Common beginner mistakes.

---

2. What Is an M-ary Tree?

An **M-ary tree** is a tree where each node can have at most **M children**.

The letter **M** means:

```
maximum number of children allowed for one node
```

For example:

Tree type	Maximum children per node
Binary tree	2

Tree type	Maximum children per node
3-ary tree	3
4-ary tree	4
5-ary tree	5
M-ary tree	M

A binary tree is just an M-ary tree where:

```
M = 2
```

---

3. Why Do We Use the Letter M?

In tree formulas, we usually use:

```
N = number of nodes
H = height of the tree
M = maximum number of children allowed per node
```

So we do not usually use `N` for the number of children, because `N` is often already used for the number of nodes.

Example:

```
In a 3-ary tree, M = 3.
In a 4-ary tree, M = 4.
In a binary tree, M = 2.
```

---

4. Normal M-ary Tree Rule

In a normal M-ary tree, each node may have:

```
0 children
1 child
2 children
```

```
...
M children
```

That means the node does **not** need to have exactly M children.

It can have **up to** M children.

Example: Normal 3-ary Tree

In a 3-ary tree, each node can have at most 3 children.

So a node may have:

```
0 children
1 child
2 children
3 children
```

All of these are allowed in a normal 3-ary tree.

---

5. Example of a 3-ary Tree

```
      A
    /  |
   B   C   D
```

Here:

- Node `A` has 3 children: `B`, `C`, and `D`.
- Nodes `B`, `C`, and `D` have 0 children.

This is a valid 3-ary tree because no node has more than 3 children.

---

6. Another Valid 3-ary Tree

```
      A
    /
   B     C
```

Here:

- Node `A` has 2 children.
- Nodes `B` and `C` have 0 children.

This is still a valid 3-ary tree.

Why?

Because a normal 3-ary tree allows **at most** 3 children.

So 2 children is allowed.

Important beginner idea:

> A 3-ary tree does not mean every node must have 3 children. It means every node can have up to 3 children.

---

7. Binary Tree vs M-ary Tree

A binary tree has only two child positions:

```
left child
right child
```

So in C++, a binary tree node can be written like this:

```cpp
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

But an M-ary tree may have many child positions.

For example, in a 3-ary tree, a node may have these child slots:

```
child 0
child 1
child 2
```

In a 4-ary tree, a node may have these child slots:

```
child 0
child 1
child 2
child 3
```

So instead of writing many separate pointer names, we usually use a vector.

---

8. Why We Use a Vector for M-ary Trees

In a binary tree, this is enough:

```
Node* left;
Node* right;
```

But in an M-ary tree, writing this would become messy:

```
Node* child1;
Node* child2;
Node* child3;
Node* child4;
Node* child5;
...
```

So we use:

```
vector<Node*> children;
```

This means:

> Store all child addresses inside one vector.

For a 3-ary tree, the vector has 3 child slots:

```
children[0]
children[1]
children[2]
```

Each slot can either:

- point to a real child node, or

- store `nullptr` if that child does not exist.

---

9. M-ary Node Structure in C++

Example for a 3-ary tree:

```cpp
#include <iostream>
#include <vector>
using namespace std;

const int M = 3;

struct Node {
    int data;
    vector<Node*> children;

    Node(int val) {
        data = val;
        children.resize(M, nullptr);
    }
};
```

---

10. Explanation of the M-ary Node Code

10.1 The M Value

```cpp
const int M = 3;
```

This means:

```
Each node can have at most 3 children.
```

So this program is working with a 3-ary tree.

10.2 Data Field

```cpp
int data;
```

This stores the value inside the node.

Example values:

```
10, 20, 30, 40
```

10.3 Children Vector

```
vector<Node*> children;
```

This stores the addresses of the child nodes.

The vector does not store only values.

It stores pointers to child nodes.

That is important because child nodes may also have their own children.

10.4 Resizing the Vector

```
children.resize(M, nullptr);
```

This creates exactly `M` child slots.

For `M = 3`, it creates:

```
children[0]
children[1]
children[2]
```

Each slot starts as `nullptr`.

That means:

```
No child exists there yet.
```

---

11. Memory View of One 3-ary Node

Suppose we create this node:

```
Node* root = new Node(10);
```

At first, it looks like this:

```
data = 10

children[0] = nullptr
children[1] = nullptr
children[2] = nullptr
```

This means node `10` currently has no children.

---

12. Adding Children to an M-ary Node

Now suppose we write:

```
root->children[0] = new Node(20);
root->children[1] = new Node(30);
root->children[2] = new Node(40);
```

This creates this tree:

```
        10
      /   |
   20     30   40
```

Meaning:

```
children[0] points to 20
children[1] points to 30
children[2] points to 40
```

Now node `10` has 3 children.

---

13. What Is a Strict M-ary Tree?

A **strict M-ary tree** is an M-ary tree where every node has either:

```
0 children
```

or:

```
M children
```

No other number of children is allowed.

So:

```
Strict M-ary tree = every node has 0 or M children
```

This is the general version of a strict binary tree.

---

14. Strict Binary Tree Review

In Phase 7, you learned strict binary trees.

A strict binary tree allows each node to have:

```
0 children or 2 children
```

Why 2?

Because a binary tree has:

```
M = 2
```

So a strict binary tree is really a strict M-ary tree where `M = 2`.

---

15. Strict 3-ary Tree Rule

For a strict 3-ary tree:

```
M = 3
```

So each node must have:

```
0 children or 3 children
```

A node with 1 child is not allowed.

A node with 2 children is not allowed.

A node with 3 children is allowed.

A node with 0 children is allowed.

---

16. Normal 3-ary Tree vs Strict 3-ary Tree

This is an important difference.

Normal 3-ary tree

A node can have:

```
0, 1, 2, or 3 children
```

Strict 3-ary tree

A node can have only:

```
0 or 3 children
```

So strict trees are more restricted.

---

17. Normal 3-ary Example That Is Not Strict

```
      A
    /
  B     C
```

Node `A` has 2 children.

This is valid as a normal 3-ary tree.

But it is not valid as a strict 3-ary tree.

Why?

Because a strict 3-ary tree allows only:

```
0 children or 3 children
```

Node `A` has 2 children.

So it breaks the strict rule.

---

18. Valid Strict 3-ary Tree Example

```
      A
    /  |
   B   C   D
```

Check each node:

Node	Number of children	Strict 3-ary valid?
A	3	Yes
B	0	Yes
C	0	Yes
D	0	Yes

Every node has either 0 or 3 children.

So this is a strict 3-ary tree.

---

19. Invalid Strict 3-ary Tree Example

```
      A
    /
  B      C
```

Check each node:

Node	Number of children	Strict 3-ary valid?
A	2	No
B	0	Yes
C	0	Yes

Only one invalid node is enough to make the whole tree not strict.

So this tree is not strict.

---

20. Main Memory Trick

Remember this:

```
Normal M-ary tree: 0 to M children allowed.
Strict M-ary tree: only 0 or M children allowed.
```

For `M = 3`:

```
Normal 3-ary: 0, 1, 2, or 3 children allowed.
Strict 3-ary: only 0 or 3 children allowed.
```

Part A: Strict M-ary Validation

21. What Does Validation Mean?

To validate a strict M-ary tree means:

> Check whether the tree follows the strict M-ary rule.

The rule is:

```
Every node must have either 0 children or M children.
```

The function should return:

```
true  -> the tree is strict
false -> the tree is not strict
```

---

22. How to Check a Node

At each node:

1. Count how many real children it has.
2. If the count is `0`, the node is valid.
3. If the count is `M`, the node is valid.
4. If the count is anything else, the node is invalid.
5. If the current node is valid, recursively check its children.

For `M = 3`:

Actual children	Strict 3-ary?
0	Yes
1	No
2	No
3	Yes

---

23. Strict M-ary Validation Code

```cpp
bool isStrictMary(Node* root) {
    if (root == nullptr) {
        return true;
    }

    int actualChildren = 0;

    for (int i = 0; i < M; i++) {
        if (root->children[i] != nullptr) {
```

```
            actualChildren++;
        }
    }

    if (actualChildren != 0 && actualChildren != M) {
        return false;
    }

    for (int i = 0; i < M; i++) {
        if (!isStrictMary(root->children[i])) {
            return false;
        }
    }

    return true;
}
```

---

24. Explanation of the Validation Code

24.1 Empty Position Case

```
if (root == nullptr) {
    return true;
}
```

If there is no node here, there is no violation.

So we return `true`.

This is similar to strict binary tree validation.

An empty child slot does not automatically make the tree wrong.

24.2 Count Actual Children

```
int actualChildren = 0;
```

This variable counts how many real child nodes exist.

At the beginning, the count is 0.

24.3 Loop Through All Child Slots

```
for (int i = 0; i < M; i++) {
    if (root->children[i] != nullptr) {
        actualChildren++;
    }
}
```

This loop checks all possible child positions.

For `M = 3`, it checks:

```
children[0]
children[1]
children[2]
```

If a child slot is not `nullptr`, that means a real child exists there.

So we increase `actualChildren`.

24.4 Apply the Strict Rule

```
if (actualChildren != 0 && actualChildren != M) {
    return false;
}
```

This means:

```
If actual children is not 0
AND
actual children is not M
then the tree is not strict.
```

For `M = 3`:

```
actualChildren = 0 -> valid
actualChildren = 1 -> invalid
actualChildren = 2 -> invalid
actualChildren = 3 -> valid
```

24.5 Recursively Check Children

```
for (int i = 0; i < M; i++) {
    if (!isStrictMary(root->children[i])) {
        return false;
    }
}
```

If the current node is valid, we still need to check all child subtrees.

Why?

Because one bad node anywhere in the tree makes the whole tree not strict.

The function calls itself on every child slot.

If any child subtree is not strict, we return `false`.

24.6 Final Return

```
return true;
```

If the current node is valid and all child subtrees are valid, the function returns `true`.

---

25. Dry Run: Strict 3-ary Tree

Use this tree:

```
       A
    /  |
   B   C   D
```

Call:

```
isStrictMary(A)
```

Step-by-step:

Step	Function call	What happens?	Result
1	`isStrictMary(A)`	A has 3 children	valid so far
2	`isStrictMary(B)`	B has 0 children	true
3	`isStrictMary(C)`	C has 0 children	true
4	`isStrictMary(D)`	D has 0 children	true
5	Back to A	all children are strict	true

Final answer:

```
The tree is strict.
```

---

26. Dry Run: Not Strict 3-ary Tree

Use this tree:

```
      A
    /
  B     C
```

Call:

```
isStrictMary(A)
```

Step-by-step:

Step	Function call	What happens?	Result
1	`isStrictMary(A)`	A has 2 children	invalid
2	Return immediately	2 is neither 0 nor 3	false

Final answer:

```
The tree is not strict.
```

Why?

Because node `A` has 2 children, but a strict 3-ary tree requires 0 or 3 children.

Part B: Strict M-ary Tree Formulas

27. Why Strict M-ary Formulas Matter

Strict M-ary trees have special formulas because their structure is restricted.

In a normal M-ary tree, nodes can have many different child counts.

But in a strict M-ary tree, every node has only:

```
0 children or M children
```

Because of this, we can predict relationships between:

- height,
- total nodes,
- internal nodes,
- external nodes,
- leaves.

---

28. Height Convention

In this curriculum, height counts edges.

A single root node has height 0.

Example:

```
A
```

Height:

```
0
```

Because there are no edges.

Example:

```
A
|
B
```

Height:

```
1
```

Because the longest path has 1 edge.

---

29. Minimum Nodes for Height H

For a strict M-ary tree:

```
Minimum nodes = M * H + 1
```

Where:

```
M = maximum number of children per internal node
H = height of the tree
```

---

30. Why Minimum Nodes = M * H + 1

To create height `H`, we need one long path downward.

But because the tree is strict, every internal node on that path must have exactly `M` children.

So each internal node on the path must create extra children to satisfy the strict rule.

That is why the minimum number of nodes grows by `M` each time height increases by 1.

Formula:

```
N = M * H + 1
```

---

31. Minimum Nodes Example: Strict 3-ary Tree, Height 2

Let:

```
M = 3
H = 2
```

Formula:

```
Minimum nodes = M * H + 1
              = 3 * 2 + 1
              = 6 + 1
              = 7
```

So a strict 3-ary tree of height 2 needs at least 7 nodes.

Example shape:

```
            A
         /    |
       B      C      D
    /  |
   E   F   G
```

Longest path:

```
A -> B -> E
```

This path has 2 edges.

So height is 2.

Count nodes:

```
A, B, C, D, E, F, G = 7 nodes
```

The formula is correct.

---

32. Maximum Nodes for Height H

For a strict M-ary tree:

```
Maximum nodes = (M^(H + 1) - 1) / (M - 1)
```

This formula means the tree is fully filled at every level.

---

33. Why Maximum Nodes Use This Formula

In a fully filled M-ary tree:

```
Level 0 has 1 node
Level 1 has M nodes
Level 2 has M^2 nodes
Level 3 has M^3 nodes
...
Level H has M^H nodes
```

So the total is:

```
1 + M + M^2 + M^3 + ... + M^H
```

This geometric sum becomes:

```
(M^(H + 1) - 1) / (M - 1)
```

---

34. Maximum Nodes Example: Strict 3-ary Tree, Height 2

Let:

```
M = 3
H = 2
```

Formula:

```
Maximum nodes = (M^(H + 1) - 1) / (M - 1)
              = (3^(2 + 1) - 1) / (3 - 1)
              = (3^3 - 1) / 2
              = (27 - 1) / 2
              = 26 / 2
              = 13
```

Level-by-level check:

```
Level 0: 1 node
Level 1: 3 nodes
Level 2: 9 nodes
```

Total:

```
1 + 3 + 9 = 13
```

So the formula is correct.

---

35. Node Range for Height H

For a strict M-ary tree with height `H`:

```
Minimum nodes = M * H + 1
Maximum nodes = (M^(H + 1) - 1) / (M - 1)
```

So:

```
M * H + 1 <= N <= (M^(H + 1) - 1) / (M - 1)
```

Example for `M = 3` and `H = 2`:

```
Minimum nodes = 3 * 2 + 1 = 7
Maximum nodes = (3^3 - 1) / 2 = 13
```

So a strict 3-ary tree of height 2 can have between:

```
7 and 13 nodes
```

---

36. Maximum Height from N Nodes

For a strict M-ary tree:

```
Maximum height = (N - 1) / M
```

This comes from the minimum-node formula.

Minimum-node formula:

```
N = M * H + 1
```

Solve for `H`:

```
N - 1 = M * H
H = (N - 1) / M
```

So:

```
Maximum height = (N - 1) / M
```

---

37. Maximum Height Example

Suppose a strict 3-ary tree has 7 nodes.

Let:

```
M = 3
N = 7
```

Formula:

```
Maximum height = (N - 1) / M
               = (7 - 1) / 3
               = 6 / 3
               = 2
```

So the maximum height is 2.

---

38. Internal and External Nodes

In a strict M-ary tree, nodes are usually divided into two groups:

```
Internal nodes
External nodes
```

---

39. Internal Node in a Strict M-ary Tree

An internal node is a node that has children.

In a strict M-ary tree, if a node has children, it must have exactly `M` children.

So:

```
Internal node = node with exactly M children
```

For a strict 3-ary tree:

```
Internal node = node with exactly 3 children
```

---

40. External Node in a Strict M-ary Tree

An external node is a leaf node.

A leaf node has no children.

So:

```
External node = leaf node = node with 0 children
```

---

41. Leaves from Internal Nodes Formula

For a strict M-ary tree:

```
E = (M - 1)I + 1
```

Where:

```
E = number of external nodes / leaves
I = number of internal nodes
M = maximum number of children per internal node
```

---

42. Why E = (M - 1)I + 1

In a strict M-ary tree, every internal node creates `M` child links.

If there are `I` internal nodes, then total child links are:

```
M * I
```

A tree with `N` nodes has:

```
N - 1 edges
```

In a strict M-ary tree:

```
N = I + E
```

Also:

```
edges = N - 1
edges = M * I
```

So:

```
N - 1 = M * I
```

Since:

```
N = I + E
```

Substitute:

```
I + E - 1 = M * I
```

Move terms:

```
E - 1 = M * I - I
E - 1 = (M - 1)I
E = (M - 1)I + 1
```

So the formula is:

```
External nodes = (M - 1) * internal nodes + 1
```

---

43. Leaves Formula Example

Suppose a strict 3-ary tree has 2 internal nodes.

Let:

```
M = 3
I = 2
```

Formula:

```
E = (M - 1)I + 1
  = (3 - 1) * 2 + 1
  = 2 * 2 + 1
```

```
= 4 + 1
= 5
```

So the tree has 5 leaves.

Check this tree:

```
            A
        /   |
       B    C     D
    /  |
   E   F   G
```

Internal nodes:

```
A, B
```

So:

```
I = 2
```

Leaves:

```
C, D, E, F, G
```

So:

```
E = 5
```

The formula is correct.

---

44. Total Nodes from Internal Nodes Formula

For a strict M-ary tree:

```
N = M * I + 1
```

Where:

```
N = total number of nodes
M = maximum children per internal node
I = internal nodes
```

---

45. Why N = M * I + 1

We know:

```
N = I + E
```

And:

```
E = (M - 1)I + 1
```

Substitute:

```
N = I + (M - 1)I + 1
```

Combine internal node terms:

```
N = M * I + 1
```

So:

```
Total nodes = M * internal nodes + 1
```

---

46. Total Nodes Example

Suppose a strict 3-ary tree has 2 internal nodes.

Let:

```
M = 3
I = 2
```

Formula:

```
N = M * I + 1
  = 3 * 2 + 1
  = 6 + 1
  = 7
```

So the tree has 7 total nodes.

Check the same tree:

```
            A
        /   |
       B    C     D
    /  |
   E   F   G
```

Total nodes:

```
A, B, C, D, E, F, G = 7
```

The formula is correct.

---

47. Formula Summary Table

Question	Strict M-ary formula	Simple meaning
Minimum nodes from height `H`	`N = M * H + 1`	Smallest strict tree with height H
Maximum nodes from height `H`	`N = (M^(H + 1) - 1) / (M - 1)`	Fully filled tree of height H
Maximum height from nodes `N`	`H = (N - 1) / M`	Tallest possible strict M-ary tree
Leaves from internal nodes `I`	`E = (M - 1)I + 1`	Number of external nodes
Total nodes from internal nodes `I`	`N = M * I + 1`	Total nodes in the strict M-ary tree

---

48. How These Formulas Connect to Strict Binary Trees

A strict binary tree is the same as a strict M-ary tree where:

```
M = 2
```

So if we use `M = 2`, the M-ary formulas become the strict binary tree formulas.

Minimum nodes

Strict M-ary formula:

```
N = M * H + 1
```

Put `M = 2`:

```
N = 2H + 1
```

This is the strict binary tree minimum-node formula.

Leaves from internal nodes

Strict M-ary formula:

```
E = (M - 1)I + 1
```

Put `M = 2`:

```
E = (2 - 1)I + 1
E = I + 1
```

This is the strict binary tree internal/external formula.

So Phase 9 generalizes Phase 7.

Part C: Complete Beginner C++ Program

49. Complete Program

```cpp
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

const int M = 3;

struct Node {
    int data;
    vector<Node*> children;

    Node(int val) {
        data = val;
        children.resize(M, nullptr);
    }
};

bool isStrictMary(Node* root) {
    if (root == nullptr) {
        return true;
    }

    int actualChildren = 0;

    for (int i = 0; i < M; i++) {
        if (root->children[i] != nullptr) {
            actualChildren++;
        }
    }

    if (actualChildren != 0 && actualChildren != M) {
        return false;
    }

    for (int i = 0; i < M; i++) {
        if (!isStrictMary(root->children[i])) {
            return false;
        }
    }

    return true;
}
```

```
int minNodesFromHeight(int h) {
    return M * h + 1;
}

int maxNodesFromHeight(int h) {
    return ((int)pow(M, h + 1) - 1) / (M - 1);
}

int maxHeightFromNodes(int n) {
    return (n - 1) / M;
}

int leavesFromInternal(int internalNodes) {
    return (M - 1) * internalNodes + 1;
}

int totalNodesFromInternal(int internalNodes) {
    return M * internalNodes + 1;
}

int main() {
    Node* root = new Node(10);

    root->children[0] = new Node(20);
    root->children[1] = new Node(30);
    root->children[2] = new Node(40);

    cout << "Strict " << M << "-ary? ";
    cout << (isStrictMary(root) ? "Yes" : "No") << endl;

    int h = 2;
    cout << "Minimum nodes for height " << h << " = "
         << minNodesFromHeight(h) << endl;

    cout << "Maximum nodes for height " << h << " = "
         << maxNodesFromHeight(h) << endl;

    int internalNodes = 2;
    cout << "Leaves from " << internalNodes << " internal nodes = "
         << leavesFromInternal(internalNodes) << endl;

    cout << "Total nodes from " << internalNodes << " internal nodes = "
         << totalNodesFromInternal(internalNodes) << endl;

    return 0;
}
```

50. Expected Output

```
Strict 3-ary? Yes
Minimum nodes for height 2 = 7
Maximum nodes for height 2 = 13
Leaves from 2 internal nodes = 5
Total nodes from 2 internal nodes = 7
```

---

51. Explanation of the Complete Program

51.1 Header Files

```
#include <iostream>
#include <vector>
#include <cmath>
```

Meaning:

Header	Why it is used
`iostream`	For input/output using `cout`
`vector`	To store child pointers
`cmath`	To use `pow()` in the maximum-node formula

51.2 M Value

```
const int M = 3;
```

This tells the program:

```
We are working with a 3-ary tree.
```

To make it a 4-ary tree, you could change it to:

```
const int M = 4;
```

51.3 Node Structure

```
struct Node {
    int data;
    vector<Node*> children;

    Node(int val) {
        data = val;
        children.resize(M, nullptr);
    }
};
```

Each node stores:

1. Its data value.
2. A vector of child pointers.

For `M = 3`, each node gets 3 child slots.

51.4 Creating the Root

```
Node* root = new Node(10);
```

This creates the root node with data value `10`.

At first, root has no children:

```
children[0] = nullptr
children[1] = nullptr
children[2] = nullptr
```

51.5 Adding Three Children

```
root->children[0] = new Node(20);
root->children[1] = new Node(30);
root->children[2] = new Node(40);
```

This creates:

```
        10
      /    |
   20     30    40
```

Now the root has exactly 3 children.

Since `M = 3`, this satisfies the strict 3-ary rule.

51.6 Checking Strictness

```
cout << (isStrictMary(root) ? "Yes" : "No") << endl;
```

This prints:

```
Yes
```

because the tree is strict.

51.7 Formula Functions

```
int minNodesFromHeight(int h) {
    return M * h + 1;
}
```

This calculates minimum nodes for height `h`.

```
int maxNodesFromHeight(int h) {
    return ((int)pow(M, h + 1) - 1) / (M - 1);
}
```

This calculates maximum nodes for height `h`.

```
int maxHeightFromNodes(int n) {
    return (n - 1) / M;
}
```

This calculates maximum height from total nodes `n`.

```
int leavesFromInternal(int internalNodes) {
    return (M - 1) * internalNodes + 1;
}
```

This calculates leaves from internal nodes.

```
int totalNodesFromInternal(int internalNodes) {
    return M * internalNodes + 1;
}
```

This calculates total nodes from internal nodes.

Part D: Common Beginner Mistakes

52. Mistake 1: Thinking Every M-ary Node Must Have M Children

Wrong idea:

```
In a 3-ary tree, every node must have 3 children.
```

Correct idea:

```
In a normal 3-ary tree, every node can have at most 3 children.
```

So a node with 0, 1, 2, or 3 children is valid in a normal 3-ary tree.

---

53. Mistake 2: Confusing Normal M-ary with Strict M-ary

Normal M-ary tree:

```
0 to M children allowed
```

Strict M-ary tree:

```
only 0 or M children allowed
```

Example for `M = 3`:

Number of children	Normal 3-ary	Strict 3-ary
0	Valid	Valid
1	Valid	Invalid
2	Valid	Invalid
3	Valid	Valid

---

54. Mistake 3: Using Binary Formulas for Every M

Wrong idea:

```
Minimum nodes = 2H + 1 for all strict trees.
```

Correct idea:

```
Minimum nodes = M * H + 1
```

The formula `2H + 1` works only when:

```
M = 2
```

That means it works for strict binary trees only.

---

55. Mistake 4: Forgetting to Count Actual Children

For validation, you must count only real child nodes.

A real child means:

```
root->children[i] != nullptr
```

A missing child means:

```
root->children[i] == nullptr
```

Do not count `nullptr` as a real child.

---

56. Mistake 5: Thinking 2 Children Is Always Strict

In a strict binary tree, 2 children is valid because:

```
M = 2
```

But in a strict 3-ary tree, 2 children is invalid because:

```
M = 3
```

For strict 3-ary, a node must have:

```
0 or 3 children
```

So 2 children is not allowed.

---

57. Mistake 6: Forgetting That Binary Trees Are M-ary Trees

A binary tree is not separate from the M-ary idea.

A binary tree is just:

```
M-ary tree with M = 2
```

So Phase 9 does not replace binary trees.

It generalizes them.

Part E: Final Memory Tricks

58. Main Memory Tricks

Concept	Memory trick
M-ary tree	At most M children
Binary tree	M-ary tree where M = 2
Normal M-ary	0 to M children allowed
Strict M-ary	only 0 or M children allowed
Internal node in strict M-ary	has exactly M children
External node in strict M-ary	has 0 children
Minimum nodes	`M * H + 1`
Maximum nodes	fully filled levels
Leaves formula	`E = (M - 1)I + 1`
Total nodes formula	`N = M * I + 1`

Part F: Practice Questions

59. Practice Question 1

Is this a valid normal 3-ary tree?

Answer

Yes.

A normal 3-ary tree allows each node to have at most 3 children.

Node `A` has 2 children, which is allowed.

---

60. Practice Question 2

Is this a strict 3-ary tree?

Answer

No.

A strict 3-ary tree allows each node to have only 0 or 3 children.

Node `A` has 2 children.

So it is not strict.

---

61. Practice Question 3

For a strict 3-ary tree with height 2, find the minimum number of nodes.

Solution

Formula:

```
Minimum nodes = M * H + 1
```

Substitute:

```
M = 3
H = 2
```

```
Minimum nodes = 3 * 2 + 1
              = 6 + 1
              = 7
```

Answer:

```
7 nodes
```

---

62. Practice Question 4

For a strict 3-ary tree with height 2, find the maximum number of nodes.

Solution

Formula:

```
Maximum nodes = (M^(H + 1) - 1) / (M - 1)
```

Substitute:

```
M = 3
H = 2
```

```
Maximum nodes = (3^(2 + 1) - 1) / (3 - 1)
              = (3^3 - 1) / 2
              = (27 - 1) / 2
              = 26 / 2
              = 13
```

Answer:

```
13 nodes
```

---

63. Practice Question 5

A strict 3-ary tree has 2 internal nodes. How many leaves does it have?

Solution

Formula:

```
E = (M - 1)I + 1
```

Substitute:

```
M = 3
I = 2
```

```
E = (3 - 1) * 2 + 1
  = 2 * 2 + 1
  = 5
```

Answer:

```
5 leaves
```

---

64. Practice Question 6

A strict 3-ary tree has 2 internal nodes. How many total nodes does it have?

Solution

Formula:

```
N = M * I + 1
```

Substitute:

```
M = 3
I = 2
```

```
N = 3 * 2 + 1
  = 6 + 1
  = 7
```

Answer:

```
7 total nodes
```

---

65. Practice Question 7

If `M = 2`, what kind of M-ary tree is it?

Answer

It is a binary tree.

A binary tree is an M-ary tree where:

```
M = 2
```

---

66. Practice Question 8

In a strict 4-ary tree, which child counts are allowed for a node?

Answer

A strict 4-ary tree allows each node to have:

```
0 children or 4 children
```

A node with 1, 2, or 3 children is not allowed.

---

67. Final Phase 9 Summary

Phase 9 teaches that binary trees are a special case of M-ary trees.

A normal M-ary tree allows each node to have at most `M` children.

A strict M-ary tree is more restricted: every node must have either `0` children or exactly `M` children.

The most important formulas are:

```
Minimum nodes from height H = M * H + 1
Maximum nodes from height H = (M^(H + 1) - 1) / (M - 1)
Maximum height from nodes N = (N - 1) / M
```

```
Leaves from internal nodes I = (M - 1)I + 1
Total nodes from internal nodes I = M * I + 1
```

The most important coding idea is:

> Use a vector of child pointers, count the non-null children, and check whether the count is 0 or M.

One-sentence memory summary:

> Normal M-ary means up to M children; strict M-ary means only 0 or M children.